



Audit summary

Semester has the beginnings of a serious platform: a substantial GitHub repository with dedicated engineering, security, monitoring, rollback, staging, and implementation documentation; a React/Vite application; two active Supabase projects; and Vercel projects for the shared application core and company site.^[1]

The highest-priority remediation is **database attack-surface control**. Production Supabase currently reports 45 tables with RLS enabled but no policy and 151 `SECURITY DEFINER` functions callable by authenticated users. Some of those may be intentional—private tables are often correctly inaccessible through RLS, while carefully designed RPCs can be valid—but this volume demands a formal allowlist, per-function review, and automated regression testing before broad production use.^[2]

Hardening scorecard

This is a baseline architectural assessment from repository structure, existing documents, platform configuration discovery, and Supabase advisor output—not a penetration test or code-level security certification.

#	Engineering dimension	Score	Current evidence	Priority remediation
1	Product architecture	4/5	Dedicated app, company site, console, platform plans, and operating-system documentation exist.	Publish a canonical architecture map with domain ownership and APIs.
2	Domain boundaries	3/5	Broad feature scope is documented, but domains need formal ownership contracts. ^[^1780]	Define bounded contexts, domain APIs, and prohibited cross-domain access.
3	Identity and authentication	3/5	Auth and role design are present in the implementation blueprint. ^[^1780]	Complete threat model, MFA policy, session lifecycle, recovery, and auth abuse tests.
4	Authorization and tenancy	3/5	Role/capability architecture is substantial, but high-risk RPC exposure needs review. ^[2]	Apply default-deny, authorization matrix, tenant-isolation tests, and RPC allowlist.
5	Database security	2/5	45 RLS-without-policy notices and 151 externally callable <code>SECURITY DEFINER</code> functions require review. ^[2]	Triage all findings; remove/revoke unintended access; verify intended private/service-only access.
6	Secrets management	3/5	<code>.gitleaks.toml</code> , <code>.env.example</code> , and secrets documentation exist.	Run recurring secret scans; rotate credentials; enforce server-only secrets.

#	Engineering dimension	Score	Current evidence	Priority remediation
7	Secure SDLC	3/5	Security, regression, rollback, staging, and monitoring documents are present.	Convert policy into required CI gates, PR templates, and release evidence.
8	Dependency supply chain	2/5	package-lock.json is present. ^[3]	Add dependency review, SBOM, vulnerability alerts, and lockfile enforcement.
9	API and input security	2/5	Large RPC surface increases validation and authorization risk. ^[2]	Schema validation, rate limits, authorization tests, replay protection, and endpoint inventory.
10	Integration architecture	3/5	Integration, LTI, and institutional architecture are documented. ^{[1780][1781]}	Create connector contracts, sandbox testing, drift detection, and reconciliation.
11	Data lineage and freshness	3/5	Source labels and freshness rules are explicitly planned. ^[^1780]	Implement field-level lineage registry, freshness SLAs, and visible stale-data fallbacks.
12	Observability	3/5	Monitoring documentation and Vercel projects are present. ^[11]	Centralize logs, traces, SLOs, synthetic checks, alert routing, and error budgets.
13	Resilience and recovery	3/5	Backup, restore, rollback, and staging documents exist.	Exercise restore drills, queue failure handling, disaster scenarios, and recovery objectives.
14	Release engineering	3/5	Separate staging/rollback documentation exists.	Require preview deploys, migration gates, canary releases, feature flags, and rollback drills.
15	Feature flags and config	3/5	Feature flags are planned; Vercel exposes feature-flag support. ^[^1780]	Establish one tenant-aware flag/config registry with expiration and auditability.
16	Performance and scalability	2/5	Architecture recognizes future scale but no measured budgets were reviewed.	Add load tests, Core Web Vitals budgets, indexing plan, queues, caching, and capacity tests.
17	Accessibility	3/5	WCAG 2.2 AA is a stated product baseline. ^[^1780]	Add automated tests plus manual keyboard, screen-reader, zoom, and mobile acceptance gates.
18	Privacy and lifecycle	3/5	Export, deletion, consent, retention, and sharing controls are planned. ^[^1780]	Formalize data inventory, retention jobs, DSR workflow, and consent enforcement tests.
19	AI governance	3/5	Source-grounded, policy-aware AI architecture is well specified. ^{[^1780][1781]}	Add prompt-injection testing, evaluation sets, audit logs, model fallback, and kill switches.
20	Tech-debt governance	3/5	Large audit and planning corpus exists.	Create a single risk register with owner, severity, due date, test, and closure evidence.

Baseline score: 57/100.

This is a promising architecture with unusually strong planning and documentation, but it

should not yet be represented as fully hardened for institution-scale sensitive data until the security, integration, observability, and release-control gaps are closed.

Immediate risks

1. Review all RLS findings

The `rls_enabled_no_policy` notice does not automatically mean a vulnerability. In a `private` schema, no policy can be deliberately secure because client roles have no access. However, the `public` tables require explicit classification:

- **Expected private/internal table:** document why no client policy is correct; confirm it is not exposed through PostgREST or direct grants.
- **Authenticated client read/write table:** create narrowly scoped RLS policies.
- **Server-only write table:** revoke client privileges and use Edge Functions or trusted server code.
- **Deprecated table:** archive or remove it after migration planning.
- **Public-read content table:** use read-only policies with no sensitive fields.

Prioritize public tables involving access gates, admins, beta programs, invitations, community safety, support tickets, payment events, SCIM credentials, leads, and GTM data.^[2]

2. Audit every SECURITY DEFINER RPC

Do not broadly revoke all execution rights: many functions may be intended as controlled client workflows. Instead, create an inventory with a required disposition for each function:

Function category	Required controls
Student self-service	Verify <code>auth.uid()</code> , tenant scope, object ownership, input validation, rate limits, audit event
Sharing / support access	Explicit consent, narrow scope, short expiry, revocation, view audit
Admin / console action	Capability check, MFA or fresh auth for high risk, dual control where needed, immutable audit
Integration control	Server-only preferred; signed workflow; replay protection; no browser service-role access
Financial / payment	Provider webhook verification, idempotency, no client-controlled final state
Community moderation	Capability check, reason required, appeals, audit trail
Read helper	Minimal fields, no hidden cross-tenant aggregation, pagination limit

For every `SECURITY DEFINER` function, set a safe `search_path`, avoid dynamic SQL unless parameterized, revoke default `PUBLIC` execution where unnecessary, and explicitly grant only the intended role. Supabase specifically flags authenticated execution because such functions can bypass RLS unless their internal authorization logic is correct.^[2]

3. Establish a formal release gate

Do not merge a migration or enable a sensitive feature unless all of the following are true:

```
[ ] Typecheck, unit tests, integration tests, and critical E2E tests pass
[ ] RLS and tenant-isolation policy tests pass
[ ] New SECURITY DEFINER functions have an approved inventory entry
[ ] Migration has a rollback or forward-fix plan
[ ] Feature flag defaults to off for production
[ ] Accessibility checks pass
[ ] Source/freshness, privacy, and error states are implemented
[ ] Observability events and dashboard exist
[ ] Deployment preview has been manually tested
[ ] Security and product owners approve high-risk changes
```

NIST's Secure Software Development Framework emphasizes integrating security practices through the lifecycle rather than treating security as a release-stage activity. ^[1821]_[1822]

Architecture map first

Build the architecture map in this sequence. Do not start with screen diagrams; start with ownership, trust boundaries, and data movement.

Layer 1: Trust boundaries

```
Public internet
  ↓
Marketing site / public tools
  ↓
Authenticated student, staff, partner, and admin clients
  ↓
API gateway / server actions / Edge Functions
  ↓
Domain services and workflow layer
  ↓
Supabase PostgreSQL, object storage, queues, audit store
  ↓
External systems: SIS, LMS, SSO, email, calendar, payments, AI providers
```

For every arrow, document:

- Authentication method.
- Authorization decision point.
- Data classifications.
- Encryption.
- Rate limit.
- Logging and audit event.
- Failure behavior.

- Retention behavior.
- Human approval requirement.

Layer 2: Core platform services

These must be shared by the app, company site, and console:

Identity and session service
Tenant and organization service
Role, capability, and consent service
Feature-flag and configuration service
Action Center and notification service
Global search and indexing service
Source lineage and freshness service
Audit and compliance event service
File/material ingestion service
Integration orchestration service
AI policy and orchestration service
Support and incident service
Analytics and experiment service

Layer 3: Business domains

Academic Path and Registration
Course and Learning
Plan, Calendar, and Time
Workspace and Files
Advising and Student Support
Campus Services and Events
Career, Skills, Portfolio, and Opportunities
Community, Groups, Mentors, and Moderation
Billing, Membership, and Entitlements
Institution Administration and Governance

Each domain needs a named owner, its own model boundaries, APIs, events, migrations, tests, observability dashboards, and deprecation policy.

Layer 4: Canonical shared objects

Do not duplicate these across modules:

Tenant
Person
Role grant
Consent grant
Term
Program
Requirement
Course and section
Plan and scenario
Action

```
Event
Resource
Meeting
Document/material
Skill
Portfolio artifact
Opportunity
Community
Source record
Integration connection
Audit event
```

Every object should contain at least:

```
id
tenantId
ownerId
createdAt
updatedAt
version
sourceSystem
sourceRecordId
sourceState
verifiedAt
syncedAt
dataClassification
retentionPolicy
```

Integration and trust layers

Integration layer design

Use a connector framework rather than implementing each vendor as a custom exception.

```
Connector
├─ Identity and owner
├─ Tenant configuration
├─ Auth method and token storage
├─ Approved scopes
├─ Capability declaration
├─ Field mapping version
├─ Sync direction
├─ Webhook contract
├─ Retry and idempotency rules
├─ Data classification map
├─ Freshness SLA
├─ Reconciliation rules
├─ Sandbox fixtures
├─ Contract tests
└─ Disconnect and deletion workflow
```

Required connector states

```
Draft
Configured
Awaiting authorization
Authorized
Validating
Active
Degraded
Paused
Reauthorization required
Failed
Disconnected
Retired
```

Integration hardening rules

- Start with read-only minimum-field access.
- Use official APIs, LTI 1.3/LTI Advantage, OAuth/OIDC, SAML, or institution-approved gateways where appropriate.
- Keep third-party credentials server-side in encrypted storage.
- Use short-lived credentials and incremental scopes.
- Verify webhook signatures, timestamps, replay protection, schema validity, and idempotency keys.
- Queue inbound events durably before processing.
- Never let a duplicate or out-of-order event create duplicate actions, payments, notifications, or records.
- Show source and last-sync time to users.
- Preserve student-owned annotations if an imported source changes.
- Provide safe official handoffs when the source is unavailable.

For learning-tool integrations, LTI is an interoperability standard designed to connect tools with institutional learning environments without separate logins; LTI Advantage expands standard services for deeper integration. ^[4] ^[5]

Trust layer design

The trust layer should be universal—not separate privacy pages and security checklists.

```
Classification
  Public · Internal · Student private · Restricted

Access
  Identity → role → capability → scope → object → consent → purpose

Evidence
  Source → freshness → confidence → explanation → audit trail
```

Control

View → share → expire → revoke → export → delete

Safety

Confirm → fresh-auth → rate-limit → log → notify → recover

For each sensitive request, evaluate:

Who is asking?
Which tenant and scope apply?
What capability is required?
Does the user own the object?
Is there a valid consent grant?
Has it expired?
Is fresh authentication needed?
What source data is being accessed?
What should be audited?
What is the safe fallback if authorization fails?

Feature flags and tenant configuration

Separate flags from configuration

A feature flag answers:

“Is this capability enabled for this context?”

Configuration answers:

“How should this enabled capability behave for this tenant?”

Use both.

Type	Example
Feature flag	<code>registration_day_mode = enabled</code>
Tenant configuration	Registration countdown uses the institution's time-ticket feed
Feature flag	<code>career_talent_pool = disabled</code>
Tenant configuration	Employer profiles require career-center approval and opt-in expiry after 180 days
Feature flag	<code>semester_ai_execute = enabled for pilot</code>
Tenant configuration	Calendar write requires fresh MFA and student preview

Recommended evaluation order

```
Global default
→ environment override
→ tenant policy
→ campus / school policy
→ department / program policy
→ cohort policy
→ role entitlement
→ user-specific pilot override
→ kill switch
```

The final result must be explainable and logged.

Minimal data model

```
feature_definitions
  key, description, type, owner, risk_tier, default_value, expires_at

feature_policies
  feature_key, scope_kind, scope_id, state, value, starts_at, ends_at, approved_by

tenant_configuration
  tenant_id, namespace, key, value, schema_version, updated_by

configuration_audit_events
  actor_id, tenant_id, change_type, before_value, after_value, reason, correlation_id

release_controls
  feature_key, environment, rollout_percent, active_kill_switch, rollback_reference
```

Rules for flags

- Every flag has an owner, purpose, risk tier, metric, expiry date, and removal ticket.
- High-risk flags require approval and an audit event.
- Every production feature can be disabled without redeployment where feasible.
- Client UI must not be the final security enforcement point; the server and database must enforce access.
- Do not place secrets or permanent authorization decisions in client-side flags.
- Use flags to stage rollout by tenant, role, cohort, and pilot—not to bypass security or testing.
- Test both enabled and disabled states.
- Remove stale flags quarterly.

90-day remediation roadmap

Days 0–14: Establish control

1. Create ARCHITECTURE.md, THREAT-MODEL.md, DATA-INVENTORY.md, INTEGRATION-CATALOG.md, and RISK-REGISTER.md.
2. Export the complete database-function inventory.
3. Triage every RLS-without-policy finding and every SECURITY DEFINER function.
4. Classify all tables and RPCs: public content, client-owned, server-only, admin-only, integration-only, deprecated.
5. Add CI checks for secret scanning, dependency vulnerabilities, migrations, typechecking, tests, and RLS regression tests.
6. Define severity levels and an incident-response process.
7. Confirm production/staging separation and ensure no production secrets appear in development environments.

Days 15–30: Secure the platform core

1. Fix or formally allowlist all database security advisor notices.
2. Add explicit grants/revocations for every RPC.
3. Add object-level authorization and tenant-isolation tests to all sensitive workflows.
4. Add rate limits and abuse controls to public forms, sign-in, search, AI, community, support, and invite endpoints.
5. Build consent and sharing-expiry enforcement.
6. Standardize audit events for login, privilege change, share, export, delete, integration change, and AI execute actions.
7. Add secure defaults for new migrations: RLS, no anonymous access, least-privilege grants, indexes, and ownership registration.

Days 31–60: Make integrations dependable

1. Implement the connector contract and connection state machine.
2. Add source lineage, field mapping registry, and data freshness metadata.
3. Build durable webhook intake, signature validation, idempotency, replay protection, and dead-letter handling.
4. Add connector contract tests and sandbox fixtures.
5. Build reconciliation jobs and the console integration-health dashboard.
6. Add graceful degraded states, official fallback links, and stale-data UX.
7. Define per-connector error budget and freshness SLA.

Days 61–90: Operationalize resilience

1. Instrument frontend, API, database, integration, queue, and AI telemetry.
2. Establish SLOs for sign-in, Today, Action Center, search, sync freshness, and support.
3. Add alert routing, on-call runbooks, and synthetic critical-journey monitoring.
4. Exercise backup restoration and incident tabletop scenarios.
5. Add rolling/canary release policies, preview verification, migration rollout gates, and tested rollback.
6. Establish performance budgets, load tests, queue capacity plans, caching strategy, and low-end-device tests.
7. Build a quarterly architecture review covering flags, integrations, permissions, dependencies, accessibility, and technical debt.

Definition of done

Semester is architecturally ready for broader institutional adoption when it can demonstrate:

- Every database table and RPC has an explicit access classification and automated authorization tests.
- Every SECURITY DEFINER function has an approved justification, constrained permissions, internal authorization checks, safe search path, audit behavior, and regression coverage.
- Every integration has a connector contract, source mapping, health state, freshness indicator, fallback, and reconciliation path.
- Every critical workflow can be monitored end to end.
- Every production release has automated checks, preview validation, feature-flag controls, migration safety, and rollback capability.
- Every sensitive action is explicitly confirmed, authorized, logged, and recoverable.
- Every tenant can configure capabilities without changing code.
- Every student-facing datum can explain its source, freshness, and limitation.
- Every major surface meets WCAG 2.2 AA acceptance criteria.^[6]
- Every major risk has an accountable owner and an evidenced closure date.

The highest-value next action is to turn the Supabase advisor output into a **Security Definer and RLS Remediation Register**. That register should be the first artifact in the architecture audit because it tells you exactly where application authority currently crosses database trust boundaries.^[2]

✱

1. <https://careerservices.upenn.edu/resources/linkedin-learning/>
2. <https://blog.khanacademy.org/new-khanmigo-interests/>
3. <https://www.khanacademy.org/about>

4. <https://www.1edtech.org/standards/lti>
5. <https://www.imsglobal.org/lti-advantage-overview>
6. <https://www.w3.org/TR/WCAG22/>
7. <https://csrc.nist.gov/projects/ssdf>
8. <https://www.perplexity.ai/search/5935cc7e-730b-431f-8ec2-48328172f793>
9. <https://www.perplexity.ai/search/e4924acb-d95d-44df-ab68-5392fed99c1c>
10. Semester_Unified_Platform_Execution_Blueprint.docx
11. Semester-Master-Implementation-Brief-v2-2.pdf
12. <https://csrc.nist.gov/pubs/sp/800/218/final>